



ОНЛАЙН-ОБРАЗОВАНИЕ

**Проверить включена ли
запись?**



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ + если все хорошо

Ставьте ☐ - если есть проблемы

Основы ORM, JPA, Hibernate как JPA-провайдер



- Узнаем кто такие ORM
- Зачем они нужны
- Какое отношение к этому имеет JPA и Hibernate
- Сегодня не очень жесть, больше теория. Где Spring – узнаем наследующем занятии
- JPA пройдем только на достаточном уровне (+-)
- Домашки нет – спокойно доделываем, готовимся к жести

Поехали!

01

Предпосылки



- Большая часть программистов, решает задачи бизнеса, автоматизирует бизнес-процессы в некоторой предметной области
- В бизнес-процессах традиционно участвуют бизнес-сущности
- Которые в процессе автоматизации находят свое отражение в классах программы
- Т.е. мы переносим сущности предметной области в программный код
- Ими там потом еще манипулируют классы сервисного слоя приложения)
- И все бы хорошо, если бы сущности не нужно было нигде хранить

- В большинстве случаев, приложения хранят данные в таблицах реляционных БД
- При этом данные разные, а операции с ними часто одни и те же (CRUD), но занимающие много места
- И естественно хочется всю эту обязанку убрать (JdbcOperations частично решает проблему), а еще лучше как-то автоматизировать операции с сущностями
- Благо, на первый взгляд, кажется, что классы отлично ложатся на табличную модель представления данных
- Но по итогу это не так (чтобы описать насколько это не так, есть даже специальный термин Impedance Mismatch)

Несоответствие между двумя подходами, объектно-ориентированным и реляционным

- Гранулярность — Количество классов и таблиц иногда разное. В одной таблице мы можем хранить данные нескольких объектов класса
- Наследование — Объектно-ориентированный мир имеет отношение наследования, но мир реляционной базы данных не поддерживает наследование
- Ассоциация — Объектно-ориентированный мир имеет ассоциативные отношения, но мир реляционных баз данных не поддерживает ассоциацию
- Идентичность — Идентичность объекта в объектно-ориентированном мире и идентичность записи в мире реляционной базы данных различны
- Навигация по данным — В объекте можно ссылаться на поля другого объекта, но с таблицами так не получится

Если какая-либо структура полностью решает это несоответствие, то эта структура называется идеальной структурой ORM



```
public class OtusStudent {  
    private long id;  
  
    private String name;  
  
    private Avatar avatar;  
  
    private List<EMail> emails;  
  
    private List<Course> courses;  
}
```

Пример: Загрузка сущности со связями через JDBC

<папка группы>/demo-projects/spring-jdbc-demo

Что там есть:

- Пример получения из БД сразу доменной сущности с использованием JdbcOperations
- В этом примере, мы с одной стороны решили N+1 проблему (точнее не допустили ее)
- А с другой не допустили появления избыточного набора данных за счет загрузки одной из связей отдельными запросами



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

02

Основы ORM, JPA, Hibernate как JPA-провайдер



- ORM (Object Relational Mapping) - позволяет отображать сущности на БД и наоборот. Помогает ей в этом JPA
- JPA (Java Persistence API) - набор интерфейсов, в основном аннотаций, для разметки классов-сущностей, который позволяет объяснять ORM, как нужно с этими классами работать
- Т.к JPA это только аннотации и интерфейсы, то для работы с ними нужна реализация. Или провайдер. Со Spring Boot по умолчанию идет Hibernate. Его мы и будем использовать

- ORM отображают сущность на таблицы БД и наоборот
- Это делается на основании предоставленных нами метаданных
- С их помощью ORM могут генерировать CRUD SQL запросы, но не только их
- Еще они могут генерировать запросы на создание таблиц и выполнять их, чтобы создать структуру БД
- В реальной работе приложения такое не понадобится, но при разработке этим можно воспользоваться

- @Entity – Говорит ORM-фреймворку, что класс является сущностью. Значит нужно считать и запомнить его метаданные, сгенерировать запросы и т.д.
- @Id – Аннотация, которой помечается поле идентификатора. Ключевое поле. Без него (или @EmbeddedId) ничего не заработает

Доп. Инфо про составные ключи:

<https://www.baeldung.com/jpa-embedded-embeddable>

<https://www.baeldung.com/jpa-composite-primary-keys>



```
@Entity // Указывает, что данный класс является сущностью
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    private long id;
    private String name;
}
```

- @Table – Вешается над классом. Задает имя таблицы, на которую будет отображаться сущность. А так же позволяет указать название схемы, уникальных полей и задать индексы*
- @Column – Вешается над полем. Задает имя поля, на которое будет отображаться имя класса, а так же ограничения данного поля*



```
@Entity // Указывает, что данный класс является сущностью
// Задаёт имя таблицы, на которую будет отображаться сущность
@Table(name = "otus_students")
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    private long id;

    // Задаёт имя и некоторые свойства поля таблицы, на которое
    // будет отображаться поле сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;
}
```

- @GeneratedValue – Указывает, что идентификатор должен генерироваться. Позволяет задать стратегию генерации и имя генератора (пишется и настраивается отдельно)
- Стратегии генерации ключей:
 - GenerationType.AUTO – автоматический выбор
 - GenerationType.IDENTITY – генерация на основе автоинкрементного поля таблицы
 - GenerationType.SEQUENCE – генерация на основе заданной последовательности (по умолчанию один на всех hibernate_sequence)
 - GenerationType.TABLE – генерация на основе заданной таблицы со значениями ключевого поля (по умолчанию одна на всех hibernate_sequence)

Доп. инфо про генерацию ключей:

<https://www.baeldung.com/hibernate-identifiers>



```
@Entity // Указывает, что данный класс является сущностью
// Задаёт имя таблицы, на которую будет отображаться сущность
@Table(name = "otus_students")
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    // Стратегия генерации идентификаторов
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    // Задаёт имя и некоторые свойства поля таблицы, на которое
    // будет отображаться поле сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;
}
```

```
@Entity
@Table(name = "otus_students")
public class OtusStudent {
    @Id
    @SequenceGenerator(initialValue = 1, allocationSize = 1, name = "idgen",
        sequenceName = "otus_students_seq")
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "idgen")
    private long id;

    @Column(name = "name", nullable = false, unique = true)
    private String name;
}
```

```
@Entity
@Table(name = "otus_students")
public class OtusStudent {
    @Id
    @GeneratedValue(generator = "custom-uuid")
    @GenericGenerator(name = "custom-uuid", strategy = "org.hibernate.id.UUIDGenerator",
        parameters = {@Parameter(name = "uuid_gen_strategy_class",
            value = "org.hibernate.id.uuid.CustomVersionOneStrategy"}})
    private UUID id;

    @Column(name = "name", nullable = false, unique = true)
    private String name;
}
```



```
@Entity
@Table(name = "otus_students")
public class OtusStudent {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @UuidGenerator
    private UUID id;

    @Column(name = "name", nullable = false, unique = true)
    private String name;
}
```

- Разметить сущности Avatar, EMail, Course, OtusStudent аннотациями @Entity и @Id
- Посмотреть структуру получившейся базы в DDL из консоли приложения или консоли H2
- Временно стереть где-нибудь @Id и проверить, что без него приложение не стартует

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git  
  
cd ./<имя группы>/spring-08-orm/orm-class-work
```

Или сразу смотрим решение в проекте orm-solution-1



- Задать новые, типичные для БД названия, таблицам и полям, соответствующим сущностям Avatar, EMail, Course, OtusStudent, с помощью аннотаций @Table и @Column
- Посмотреть структуру получившейся базы в DDL из консоли приложения или консоли H2
- Обратить внимание на изменившиеся имена

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ./<имя группы>/spring-08-orm/orm-class-work
```

Или сразу смотрим решение в проекте orm-solution-2



- Задать для идентификаторов сущностей Avatar, EMail, Course, OtusStudent стратегию генерации IDENTITY
- Посмотреть структуру получившейся базы в DDL из консоли приложения или консоли H2
- Обратить внимание на поля идентификаторов

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git  
  
cd ./<имя группы>/spring-08-orm/orm-class-work
```

Или сразу смотрим решение в проекте orm-solution-3



- Бывают следующих видов:
- @OneToOne
- @OneToMany
- @ManyToOne
- @ManyToMany
- Имеют общие атрибуты:
 - targetEntity
 - cascade
 - fetch
 - orphanRemoval*
 - mappedBy**

* – только у @OneTo* аннотаций

** – нет у @ManyToOne аннотации



- targetEntity – класс сущности на том конце связи
- Нужно только если аннотация висит над полем, из которого нельзя рефлексией вывести тип
- Object, List, Set, например. Тут надо указать
- У условного Avatar, List<Email> - не надо. Тут тип класс сущности-связи можно добыть легко

- cascade – позволяет указать какие операции будут каскадно влиять на связанные сущности
- Возможные значения: ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH
- Если, например аннотация связи со включенным cascade = {ALL, REMOVE}, висит над полем с аватаром, то при удалении студента, удалится и аватар, если ALL или PERSIST, то сохранится вместе с родителем, а соответственно если ALL или MERGE, то обновиться
- Ставить по умолчанию ALL, плохая идея. Особенно, если сущность на том конце связи, самостоятельная
- Значение по умолчанию отсутствует. Т.е. каскадные операции отключены

- Отвечает за тип загрузки связи при загрузке основной сущности
- EAGER – жадная загрузка. Когда загружается основная сущность или их список, для нее грузятся и EAGER-связи
- LAZY (рекомендованный вариант) – ленивая загрузка. Когда загружается основная сущность или их список, LAZY-связи не грузятся
- Вместо них для полей, не являющимися коллекциями в сущность проставляется пустой прокси, имеющий id связи
- А для полей-коллекций, проставляется специальная, изначально пустая, коллекция
- В момент, когда мы обратимся к свойствам такой связи, прокси или коллекция сделает запрос в БД и заполнит себя реальными значениями
- Но только если мы это сделали в той же сессии, в которой грузилась основная сущность. Иначе получим `LazyInitializationException`

- orphanRemoval* – если задано, то при “отвязывании” дочерней сущности от родительской, она будет удалена
- Например, при удалении сущности из вложенного списка внутри родительской сущности
- Или запуске поля связи
- Для самостоятельных сущностей-связей также не лучше указывать

* – только у @OneTo* аннотаций

- Связи бывают однонаправленные и двунаправленные
- Однонаправленные – когда в одной сущности есть ссылка на связанную, а в другой нет (аватар есть в студенте, а в аватаре студента нет)
- Двунаправленные – когда в обеих сущностях есть ссылка на связанную (аватар есть в студенте и в аватаре есть студент)
- mappedBy – указывает фреймворку на владельца двунаправленной связи и для такого типа связи обязателен к указанию
- С двунаправленными связями сопряжено большое количество проблем, если их настроить не правильно (переполнение стека при сериализации, equals/hashCode/toString, проблемы при работе с каскадными операциями и т.д.)
- + из-за них часто добавляют в сущности то, чего там не должно быть согласно домену
- Так что лучше их использовать четко представляя зачем это нужно

** – нет у @ManyToOne аннотации

- @OneToOne – Позволяет создавать связи между сущностями вида один-к-одному. Это значит, что для каждой родительской сущности существует только одна дочерняя сущность и наоборот
- Значение атрибута fetch по умолчанию – EAGER
- Аннотация @JoinColumn означает связующий столбец в таблице родительской сущности



```
@Entity // Указывает, что данный класс является сущностью
// Задаёт имя таблицы, на которую будет отображаться сущность
@Table(name = "otus_students")
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    // Стратегия генерации идентификаторов
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    // Задаёт имя и некоторые свойства поля таблицы, на которое будет
    // отображаться поле сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;

    // Указывает на связь между таблицами "один к одному"
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    // Задаёт поле, по которому происходит объединение с таблицей
    // для хранения связанной сущности
    @JoinColumn(name = "avatar_id")
    private Avatar avatar;
}
```

В сущностях, участвующих в ДЗ курса НЕТ связи One-to-One

(ставим "+" если запомнили этот
момент 😊)

- Связать сущности OtusStudent и Avatar с помощью @OneToOne и @JoinColumn
- Посмотреть структуру получившейся базы в DDL из консоли приложения или консоли H2
- Обратить внимание на поля внешние ключи и поля в таблице студентов

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git  
  
cd ./<имя группы>/spring-08-orm/orm-class-work
```

Или сразу смотрим решение в проекте orm-solution-4



Упражнение 5

- Привести **связи** в сущностях OtusStudent и Avatar к указанному справа виду
- Посмотреть структуру таблиц аватаров и студентов в DDL из консоли приложения
- Или можно посмотреть результат в пакете solution5



```
@Entity
public class OtusStudent {
    // ...
    @OneToOne
    private Avatar avatar;
    // ...
}

@Entity
public class Avatar {
    // ...
    @OneToOne
    private OtusStudent student;
}
```

- Привести **связи** в сущностях OtusStudent и Avatar к указанному справа виду
- Посмотреть структуру таблиц аватаров и студентов в DDL из консоли приложения
- Или можно посмотреть результат в пакете solution6



```
@Entity
public class OtusStudent {
    // ...
    @OneToOne (mappedBy="student")
    private Avatar avatar;
    // ...
}
```

```
@Entity
public class Avatar {
    // ...
    @OneToOne
    private OtusStudent student;
}
```


- Привести **связи** в сущностях OtusStudent и Avatar к указанному справа виду
- Посмотреть структуру таблиц аватаров и студентов в DDL из консоли приложения
- Или можно посмотреть результат в пакете solution7



@Entity

```
public class OtusStudent {
```

```
// ...
```

@OneToOne

```
private Avatar avatar;
```

```
// ...
```

```
}
```

@Entity

```
public class Avatar {
```

```
// ...
```

@OneToOne (mappedBy="avatar")

```
private OtusStudent student;
```

```
}
```

- @OneToMany – Позволяет создавать связи между сущностями вида один-ко-многим. Это значит, что для каждой родительской сущности может быть много дочерних, но у каждой дочерней только одна родительская сущность
- Значение атрибута fetch по умолчанию – LAZY
- Аннотация @JoinColumn означает связующий столбец в таблице дочерних сущностей
- Так же связь может осуществляться через таблицу связей (@JoinTable)



- Без mappedBy ORM считает, что у него просто **две связи** в разные стороны, **а не одна двунаправленная**
- mappedBy указывает владельца связи, т.е. в какой таблице будет связующий столбец
- При двунаправленных связях и включенных каскадных операциях **у каждой сущности должна** (должны если список) **быть ссылка на связанную**
- В данном случае, у студента должна быть ссылка на аватар, а у аватара ссылка на студента



```
@Entity
public class OtusStudent {
    @OneToOne
    private Avatar avatar;
}

@Entity
public class Avatar {
    @OneToOne
    private OtusStudent student;
}

create table avatars (
    id bigint generated by default as identity,
    photo_url varchar(255),
    student_id bigint,
    primary key (id)
)

create table students (
    id bigint generated by default as identity,
    student_name varchar(255),
    avatar_id bigint,
    primary key (id)
)
```

```
@Entity // Указывает, что данный класс является сущностью
@Table(name = "otus_students") // Задаст имя таблицы, на которую будет отображаться сущность
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Стратегия генерации идентификаторов
    private long id;

    // Задаст имя и некоторые свойства поля таблицы, на которое будет отображаться поле
    // сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;

    // Указывает на связь между таблицами "один к одному"
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    // Задаст поле, по которому происходит объединение с таблицей для хранения связанной
    // сущности
    @JoinColumn(name = "avatar_id")
    private Avatar avatar;

    // Указывает на связь между таблицами "один ко многим"
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true)
    @JoinColumn(name = "student_id")
    private List<EMail> emails;
}
```

- @ManyToMany – Позволяет создавать связи между сущностями вида многие-ко-многим. Это значит, что для каждой родительской сущности может быть много дочерних, но и у каждой дочерней может соответствовать несколько родительских сущностей
- Значение атрибута fetch по умолчанию – LAZY
- Связь осуществляется через таблицу связей, с помощью @JoinTable



```
@Entity // Указывает, что данный класс является сущностью
@Table(name = "otus_students") // Задаст имя таблицы, на которую будет отображаться сущность
public class OtusStudent {
    @Id // Позволяет указать какое поле является идентификатором
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Стратегия генерации идентификаторов
    private long id;

    // Задаст имя и некоторые свойства поля таблицы, на которое будет отображаться поле сущности
    @Column(name = "name", nullable = false, unique = true)
    private String name;

    // Указывает на связь между таблицами "один к одному"
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    // Задаст поле, по которому происходит объединение с таблицей для хранения связанной сущности
    @JoinColumn(name = "avatar_id")
    private Avatar avatar;

    // Указывает на связь между таблицами "один ко многим"
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER, orphanRemoval = true)
    @JoinColumn(name = "student_id")
    private List<EMail> emails;

    // Указывает на связь между таблицами "многие ко многим"
    @ManyToMany(fetch = FetchType.LAZY, cascade = CascadeType.PERSIST)
    // Задаст таблицу связей между таблицами для хранения родительской и связанной сущностью
    @JoinTable(name = "student_courses", joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private List<Course> courses;
}
```

- Связать сущности OtusStudent и EMail с помощью @OneToMany и @JoinColumn
- Посмотреть структуру получившейся базы в DDL из консоли приложения или консоли H2
- Обратить внимание на поля внешние ключи и поля в таблице email-ов

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ./<имя группы>/spring-08-orm/orm-class-work
```

Или сразу смотрим решение в проекте orm-solution-8



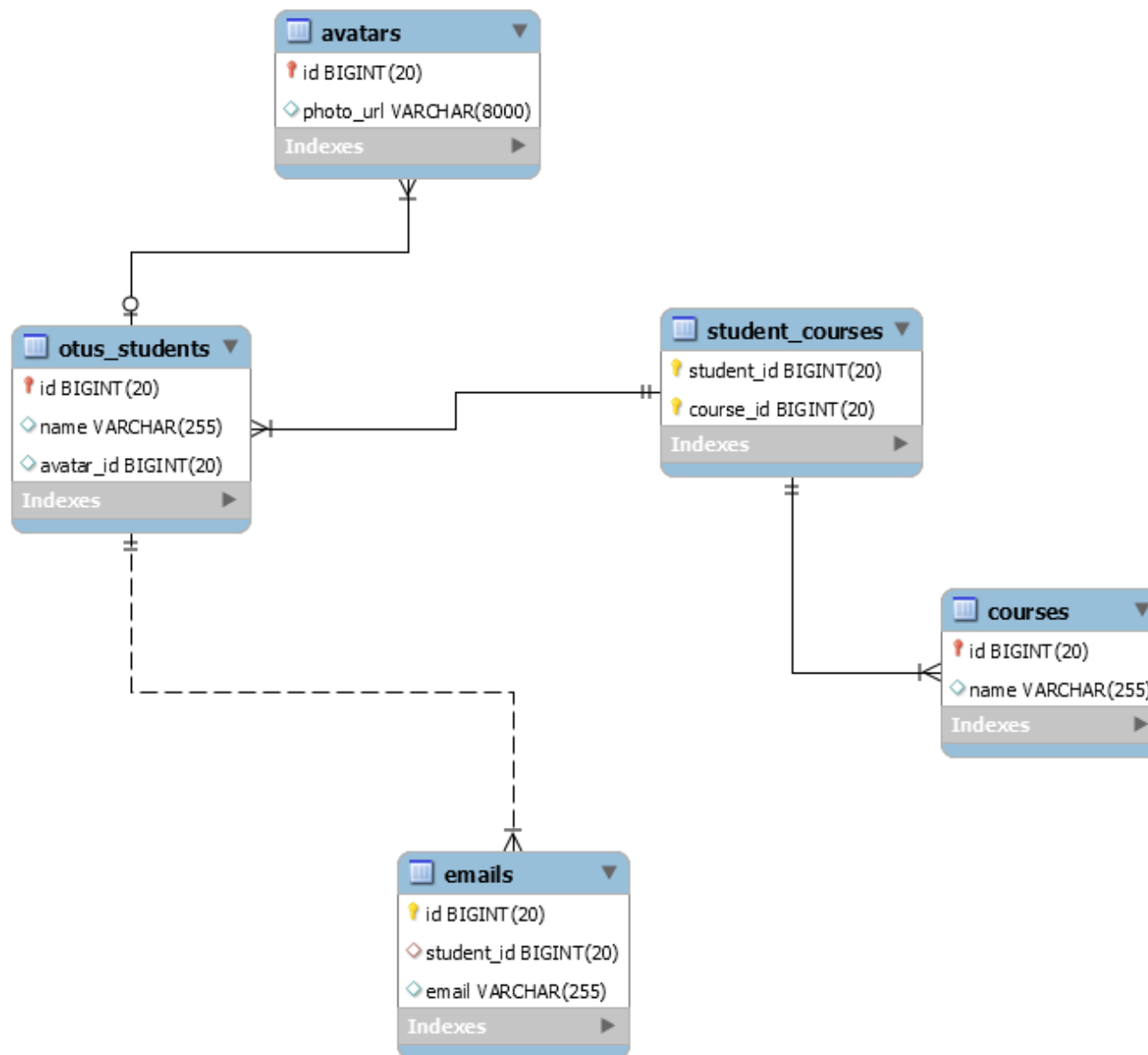
- Связать сущности OtusStudent и Course с помощью @ManyToMany и @JoinTable
- Посмотреть структуру получившейся базы в DDL из консоли приложения или консоли H2
- Обратить внимание новую таблицу связей, ее поля и внешние ключи

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ./<имя группы>/spring-08-orm/orm-class-work
```

Или сразу смотрим решение в проекте orm-solution-final





Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

Демо (Наследование)



Пример: Другой взгляд на маппинг (MyBatis)

<папка группы>/demo-projects/mybatis-demo

Что там есть:

- Пример получения из БД сразу доменной сущности с использованием MyBatis
- Тут разметка висит не над сущностью, а над интерфейсом репозитория
- Реализацию которого мы не пишем, за нас это делает MyBatis



03

**Продолжение
следует...**

- Узнаем, что такое Persistence Context и как им пользоваться
- Какие состояния может по отношению к нему принимать сущность
- Посмотрим, как делать простые операции над сущностями с помощью EntityManager
- А так же делать JPQL запросы
- Решать типовые проблемы при работе с БД, через ORM

**Пожалуйста, пройдите
опрос**

Спасибо за внимание!

ЖРА Вам!

